

CNN and RNNs

Nagesh

May 2026

”Wisom without compassion is barren Compassion without wisdom is blindness”

Contents

CNNs and RNNs as Regularized MLPs	3
1. MLP as a Fully Connected Transformation	3
Problem with MLPs on Images	4
2. CNN as Spatially Regularized MLP	4
Convolution Operation	4
CNN as Parameter Sharing	5
Comparison with MLP	5
Parameter Count Reduction	5
Geometric Interpretation of CNN	6
Stride in CNN	6
Effect of Stride	6
Padding	7
Padded Input	7
Why Padding is Needed	7
Types of Padding	7
Valid Padding	7
Same Padding	7
Reflection Padding	7
Circular Padding	8

Pooling Operations	8
Max Pooling	8
Average Pooling	8
Global Average Pooling	8
Interpretation of Pooling	8
CNN Pipeline	9
3. RNN as Temporally Regularized MLP	9
RNN Parameter Sharing	9
RNN as Constrained MLP	10
Why Temporal Sharing Helps	10
CNN vs RNN Parameter Sharing	10
Unified Interpretation	11
Inductive Bias	11
Connection to Regularization	11
Key Insight	12

CNNs and RNNs as Regularized MLPs

A Multi-Layer Perceptron (MLP) is the most general dense neural network.

However, MLPs do not exploit structural priors present in data.

Examples:

- Images have spatial locality
- Sequences have temporal structure

CNNs and RNNs can be understood as:

Regularized / constrained MLPs with parameter sharing

CNNs share parameters across space.

RNNs share parameters across time.

This drastically reduces the number of learnable parameters and introduces useful inductive biases.

1. MLP as a Fully Connected Transformation

Suppose:

$$x \in \mathbb{R}^d$$

An MLP layer computes:

$$h = \sigma(Wx + b)$$

where:

$$W \in \mathbb{R}^{m \times d}$$

Every output neuron connects to every input neuron.

Thus:

$$h_i = \sigma \left(\sum_{j=1}^d W_{ij} x_j + b_i \right)$$

Total parameters:

$$md$$

No spatial or temporal structure is assumed.

Problem with MLPs on Images

Suppose image:

$$X \in \mathbb{R}^{H \times W}$$

Flattening gives:

$$x \in \mathbb{R}^{HW}$$

For a modest image:

$$256 \times 256$$

input dimension becomes:

$$65536$$

A fully connected layer becomes extremely expensive.

Also:

- nearby pixels are correlated
- same visual pattern may appear anywhere

MLPs ignore these properties.

2. CNN as Spatially Regularized MLP

CNN introduces two assumptions:

1. Local connectivity
2. Parameter sharing across space

Convolution Operation

Suppose:

$$X \in \mathbb{R}^{H \times W}$$

and kernel:

$$K \in \mathbb{R}^{k \times k}$$

Convolution:

$$Y(i, j) = \sum_{u=1}^k \sum_{v=1}^k K(u, v) X(i + u, j + v)$$

This is a local weighted sum.

CNN as Parameter Sharing

Observe carefully:
the same kernel:

$$K(u, v)$$

is used at every spatial location.
Thus weights are shared globally across space.
This is equivalent to:

A constrained MLP with tied weights

Comparison with MLP

MLP:

$$h_i = \sum_j W_{ij} x_j$$

Every output neuron has different weights.
CNN:

$$Y(i, j) = \sum_{u, v} K(u, v) X(i + u, j + v)$$

Same weights reused everywhere.
Thus CNN imposes translational symmetry.

Parameter Count Reduction

MLP:

$$\mathcal{O}(H^2 W^2)$$

CNN:

$$\mathcal{O}(k^2)$$

Huge reduction in parameters.
This acts as regularization.

Geometric Interpretation of CNN

CNN assumes:

Patterns are spatially reusable.

Example:

- edge detector
- texture detector
- corner detector

can appear anywhere in image.

Thus the same kernel is reused everywhere.

Stride in CNN

Stride determines movement of kernel.

Suppose stride:

$$s$$

Then convolution becomes:

$$Y(i, j) = \sum_{u, v} K(u, v) X(is + u, js + v)$$

Effect of Stride

Larger stride:

- reduces spatial resolution
- increases receptive field
- lowers computation

Output size:

$$H_{\text{out}} = \frac{H - k + 2p}{s} + 1$$

$$W_{\text{out}} = \frac{W - k + 2p}{s} + 1$$

where:

- k = kernel size
- p = padding
- s = stride

Padding

Convolution reduces image size.

Padding adds artificial boundary values.

Typically zero padding.

Padded Input

Original image:

$$X \in \mathbb{R}^{H \times W}$$

Padded image:

$$X_p \in \mathbb{R}^{(H+2p) \times (W+2p)}$$

Why Padding is Needed

Padding helps:

- preserve spatial size
- avoid edge information loss
- allow deeper networks

Types of Padding

Valid Padding

No padding:

$$p = 0$$

Output shrinks.

Same Padding

Choose:

$$p = \frac{k-1}{2}$$

to preserve image size.

Reflection Padding

Boundary values mirrored.

Circular Padding

Periodic boundary conditions.

Used in Fourier methods.

Pooling Operations

Pooling reduces spatial resolution.

Max Pooling

For local region:

$$R$$

output:

$$Y = \max_{x \in R} x$$

Keeps strongest activation.

Average Pooling

$$Y = \frac{1}{|R|} \sum_{x \in R} x$$

Smooth averaging.

Global Average Pooling

Average entire feature map:

$$Y_c = \frac{1}{HW} \sum_{i,j} X_c(i,j)$$

Used in modern CNNs.

Interpretation of Pooling

Pooling introduces:

Spatial invariance

Small translations do not drastically change representation.

CNN Pipeline

Typical CNN:

Input $\rightarrow$ Conv $\rightarrow$ Nonlinearity $\rightarrow$ Pooling $\rightarrow$ Conv $\rightarrow \dots$

Hierarchical feature learning:

- edges
- textures
- object parts
- objects

3. RNN as Temporally Regularized MLP

Now consider sequential data:

$$X = \{x_1, x_2, \dots, x_T\}$$

A naive MLP would require:

$$f_t(x_1, \dots, x_t)$$

with different parameters for every time step.

This is inefficient.

RNN Parameter Sharing

RNN introduces:

Parameter sharing across time

Define hidden state:

$$h_t$$

Update rule:

$$h_t = \sigma(W_h h_{t-1} + W_x x_t + b)$$

Output:

$$y_t = W_o h_t$$

Same matrices:

$$W_h, W_x, W_o$$

are reused at every time step.

RNN as Constrained MLP

Unrolled RNN:

$$h_1 = \sigma(W_h h_0 + W_x x_1)$$

$$h_2 = \sigma(W_h h_1 + W_x x_2)$$

$$h_3 = \sigma(W_h h_2 + W_x x_3)$$

Observe:

same transformation repeated recursively.

Thus RNN is:

An MLP with tied temporal weights

Why Temporal Sharing Helps

Temporal parameter sharing assumes:

Dynamics are time-invariant.

Meaning:

same rule governs sequence evolution across all times.

Examples:

- language grammar
- dynamical systems
- time series

CNN vs RNN Parameter Sharing

CNN:

share across space

RNN:

share across time

Both reduce parameters and improve generalization.

Unified Interpretation

MLP:

$$h = \sigma(Wx)$$

CNN:

$$h(i, j) = \sum_{u, v} K(u, v) X(i + u, j + v)$$

(shared across space)

RNN:

$$h_t = \sigma(W_h h_{t-1} + W_x x_t)$$

(shared across time)

Thus:

CNNs and RNNs are structured MLPs with symmetry constraints

Inductive Bias

Parameter sharing introduces inductive bias.

CNN assumes:

Translation symmetry

RNN assumes:

Temporal stationarity

These biases improve learning efficiency.

Connection to Regularization

Parameter sharing reduces effective degrees of freedom.

Thus:

shared weights $\Rightarrow$ lower model complexity

Hence CNNs and RNNs can be viewed as:

Regularized versions of dense MLPs

with strong structural assumptions.

Key Insight

Deep learning architectures differ mainly in:

- how parameters are shared
- what symmetry assumptions are imposed
- how representations are structured

MLP:

No structure

CNN:

Spatial structure

RNN:

Temporal structure

Transformers:

Relational interaction structure